# Design Patterns

Saturday, 10 January 2026 10:31 AM

## What are design patterns?

→ reusable solutions to common software design problems.
→ clean & maintainable code.
→ flexibility & scalability.

### Benefits

→ Provide standard, proven solutions.
→ Help follow best practices.
→ Improve code readability & maintainability.
→ Follow DRY → Don't Repeat Yourself.

### Examples

→ Singleton
→ Factory
→ Decorator
→ Observer
→ Facade.

### Limitations

When to avoid using design patterns?

→ Over-engineering :- Don't use design patterns just for the sake of using them

→ Tight deadlines :-

→ Misapplication :- To avoid misapplication you should learn & understand about the pattern.

### Types of design patterns

→ Creational Design pattern
↳ deal with object creation in a flexible & efficient manner.

→ Singleton
→ Factory

→ Builder
→ Abstract Factory

→ Structural Design Patterns
↳ how classes & objects are combined to form larger structures.

- Decorator
- Facade
- Adapter
- Bridge

→ Behavioral patterns
↳ how objects communicate & distribute responsibilities.

Command
→ Observer (forms basis of Pub-Sub model)
→ State
→ Strategy

# Creational Design patterns

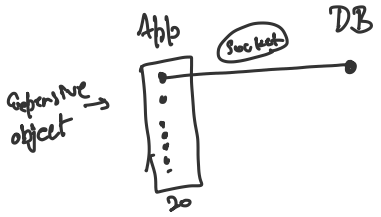
→ Focus on the process of object creation
→ Help make system independent of object creation

Singleton
most famous design pattern
It ensures that only one object instance exists into the system.

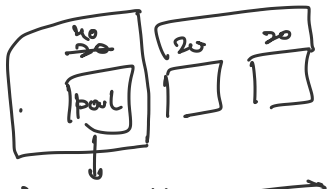
Why?
↳ why singleton is needed?
when object creation is very expensive on resources

Example:
Thread pool (shared)

Connection pool




Service




Service → query → connection → DB

## Implementation

class
↳ default constructor
↳ allowing us to create objects.

Step 1: Create / make the constructor as private

Q: If object cannot be created from outside, how will we create it?

(MyClass obj = new MyClass();) ✗

Step 2: Create a static method to get the instance.

Step 3: Create a static instance variable.

## Ways of creation

→ Eager initialisation

Prost
- Easy to implement
- Thread-safe

Cons: • Object will be created at class load even if it is not needed.

→ Simple Lazy

Pros:- • Somewhat easy to implement.
• Object created on first call of getInstance

Cons: not thread-safe.

```
t1, t2                                  1000 t1, t2
1   getInstance() ] → synchronized. ]
                                  t1 ↓ ]→
2   t1, t2
    if (INSTANCE == null) {
3 →   t1, t2
      INSTANCE = new Singleton(); ]
4   }

5   return INSTANCE;
```

(public synchronized static getInstance()) — method signature

Pros → Thread safe.
→ Easy to implement thread-safe approach

Cons → Bad for performance in heavy traffic.

Cases:
↳ we are adding synchronized to whole method when it is needed for a single statement.

solution: add synchronized block around the statement.

for Python, it will be threading.Lock.

```
t1, t2 ---1000
getInstance()                    { double-null check solution }
{ 1
  t1, t2
```

```
2   if (INSTANCE == null)
        synchronized (Singleton.class) {
3           if (INSTANCE == null) {
4               INSTANCE = new Singleton();
            }
        }
    }
5   return INSTANCE;
}
```

---

```
if (INSTANCE != null) {
    return INSTANCE;
}
synchronized (Singleton.class) {
    if (INSTANCE != null)
        return INSTANCE;
    }
    INSTANCE = new Sington();
}
```